Module 1 – System Management

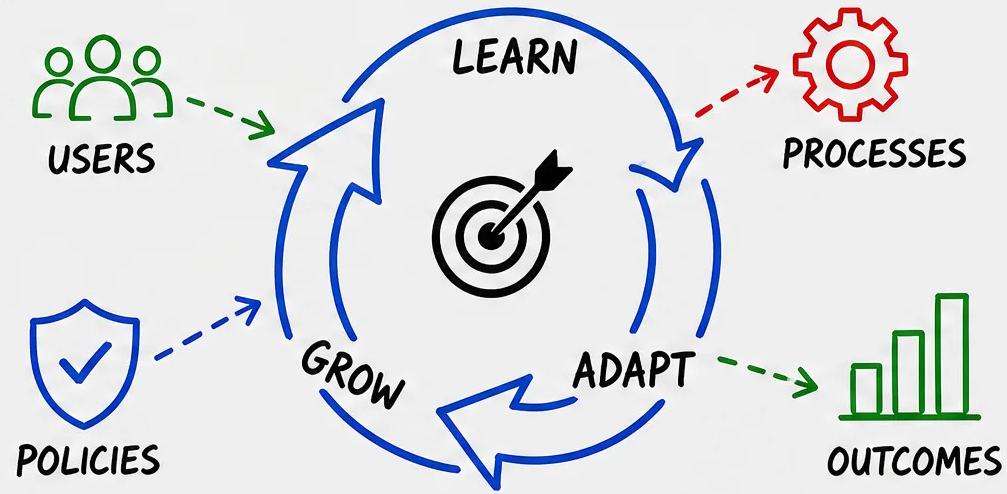


Architecture before clicks.

ProTech
protechtraining.com



Great administration begins with a **shared mental model.**

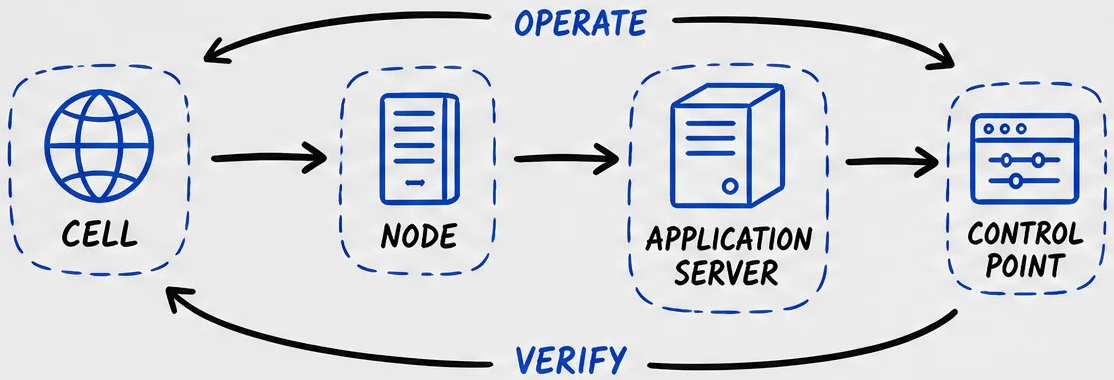


What This Module Covers — and What It Does Not

2

Define the module scope and what is explicitly excluded.

✓ This Module Covers



- ✓ Architecture before clicks
- ✓ Core administration tasks (configure & operate)
- ✓ Verify and confirm successful configuration

✗ This Module Does Not Cover

- ✗ Troubleshooting or root cause analysis
- ✗ Performance tuning or capacity planning
- ✗ Advanced customization or scripting
- ✗ External integrations or third-party products
- ✗ Disaster recovery implementation

Know Your Boundaries Before Acting.

The Coordinates: Profile, Node, Cell, Server

3

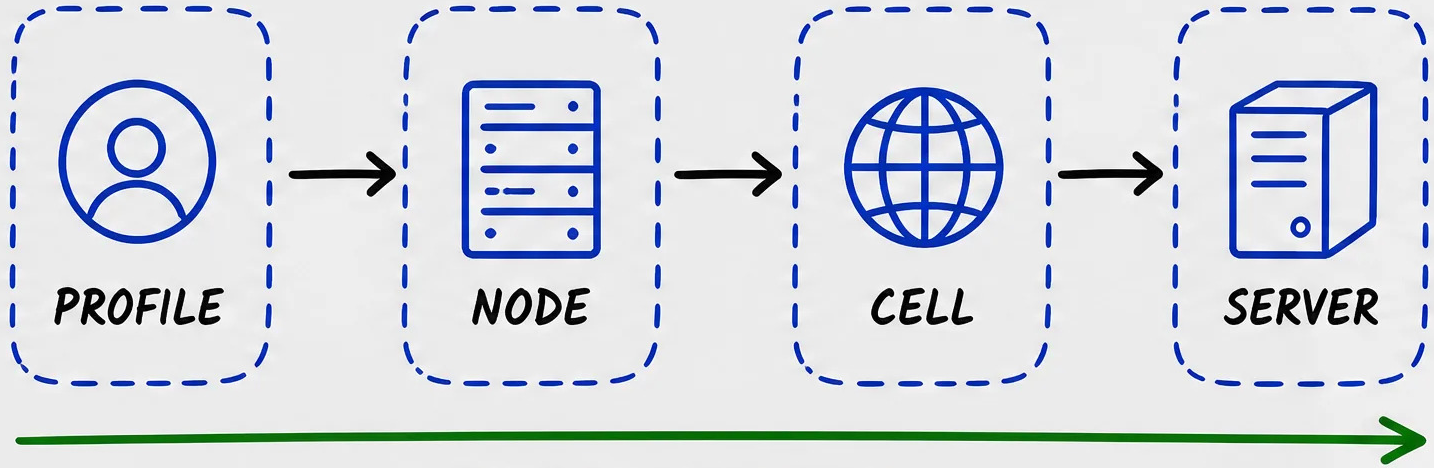
THE HIERARCHY CONTROLS THE CONFIGURATION PATH.

PROFILE: A user or group container that holds configuration.

NODE: A server instance that hosts applications.

CELL: A logical grouping of nodes that share a common configuration.

SERVER: The application server that runs and delivers services.



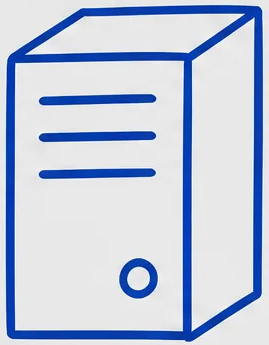
RISK: MISCONFIGURATION IMPACTS AVAILABILITY.

Who Manages What?

Distinguishing Node Agents from the Deployment Manager

Deployment Manager

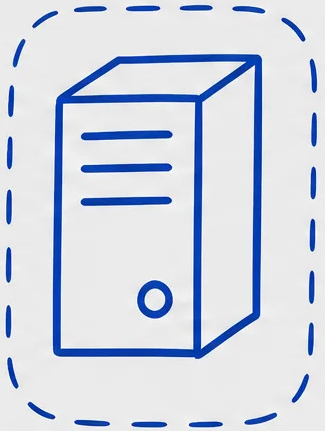
(Decides)



- Decides WHAT to deploy
- Maintains desired state
- Plans & schedules changes
- Monitors & evaluates health
- Coordinated at the cell level

Node Agent

(Acts)



- Executes WHAT is decided
- Applies configuration changes
- Starts, stops, restarts services
- Reports status & logs
- Act at the node level

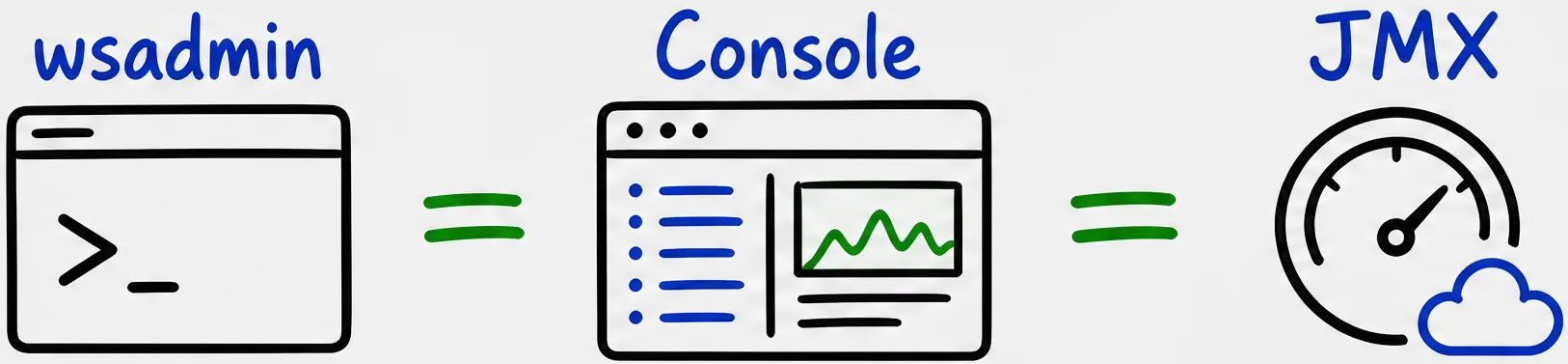


The Deployment Manager decides; the Node Agent acts.



Choose the Control Surface

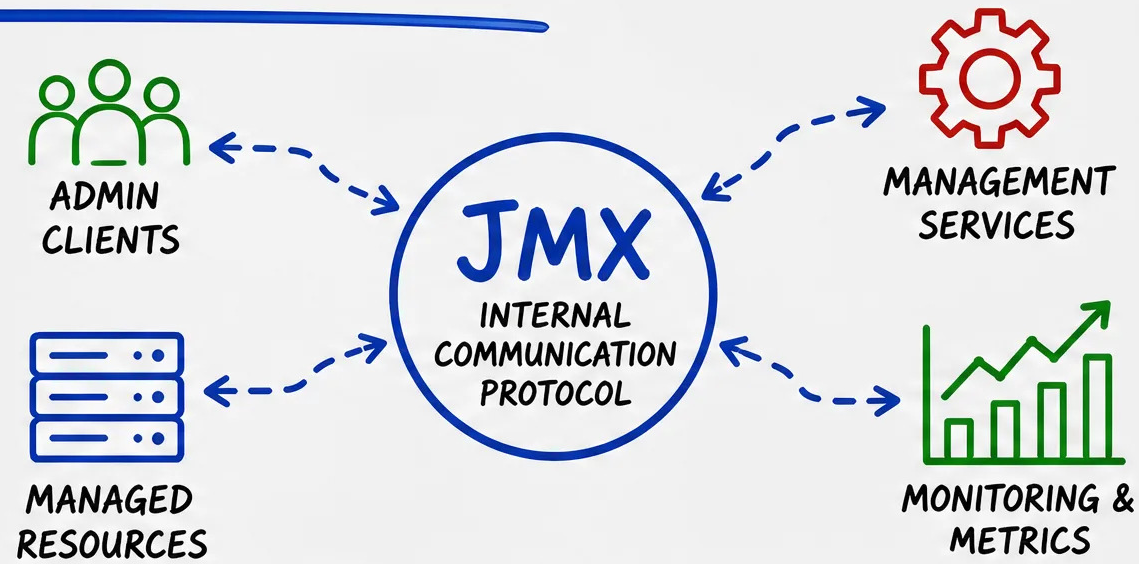
wsadmin, the console, and JMX
are equivalent control surfaces.



The console and the script are
just different doors to the same data.

JMX Is the Management Backbone

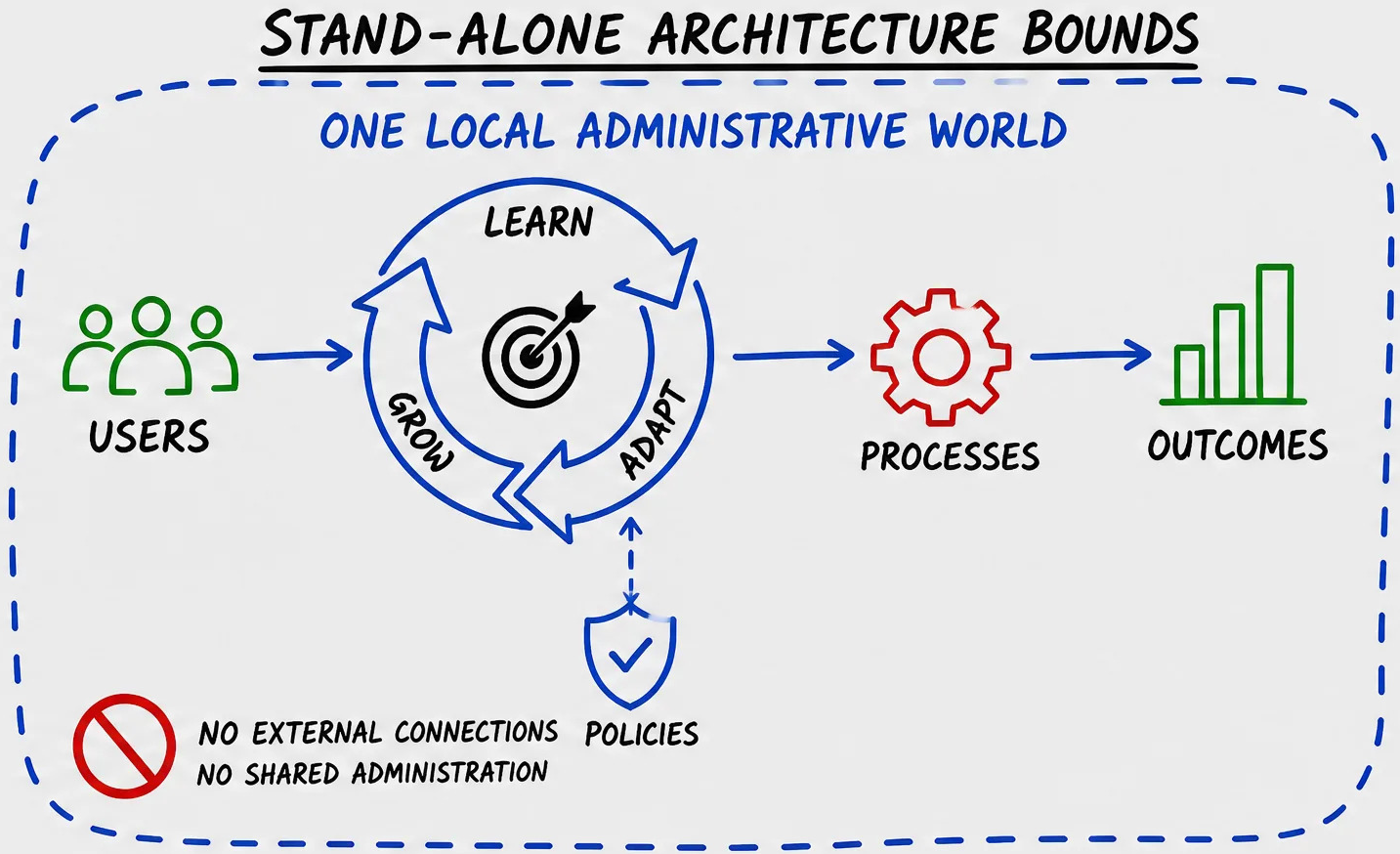
JMX is the internal communication protocol for administration.



JMX is the language of the entire WebSphere administrative fabric.

Stand-Alone: One Local Administrative World

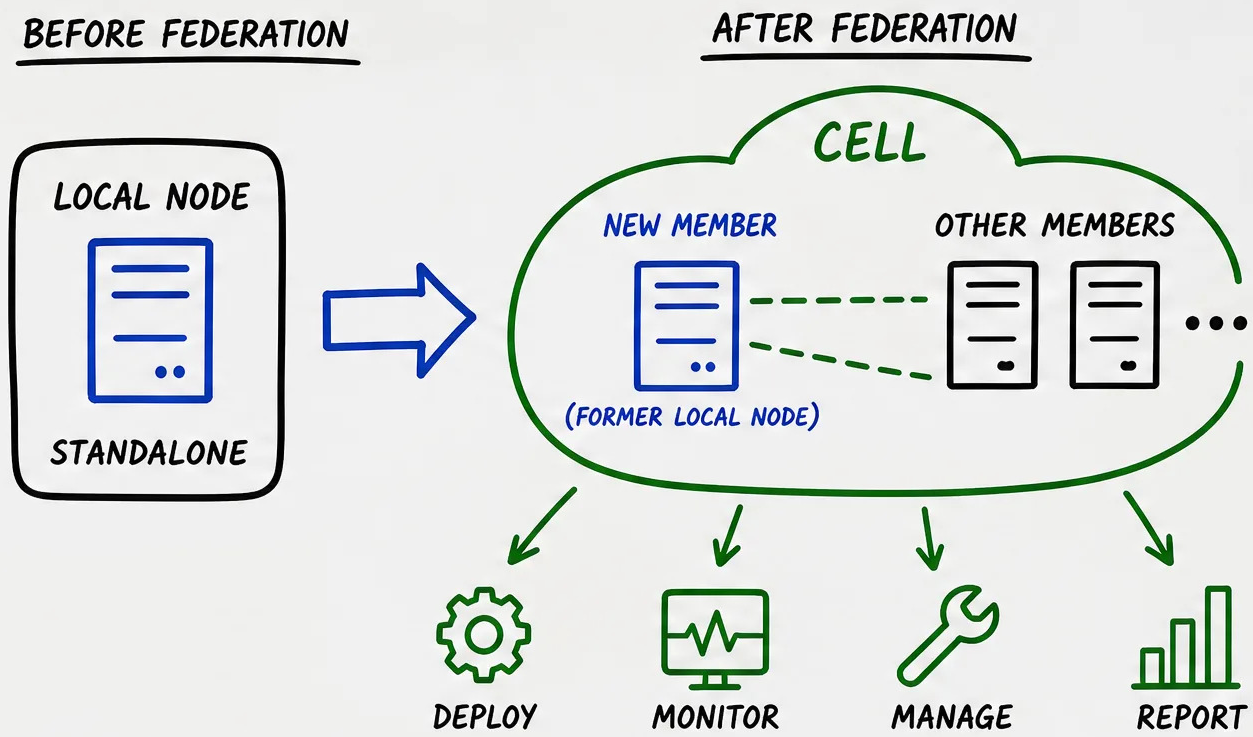
Stand-alone means complete independence, but also isolated administration.



Federation: From Local Node to Cell Member

WHAT HAPPENS DURING NODE FEDERATION

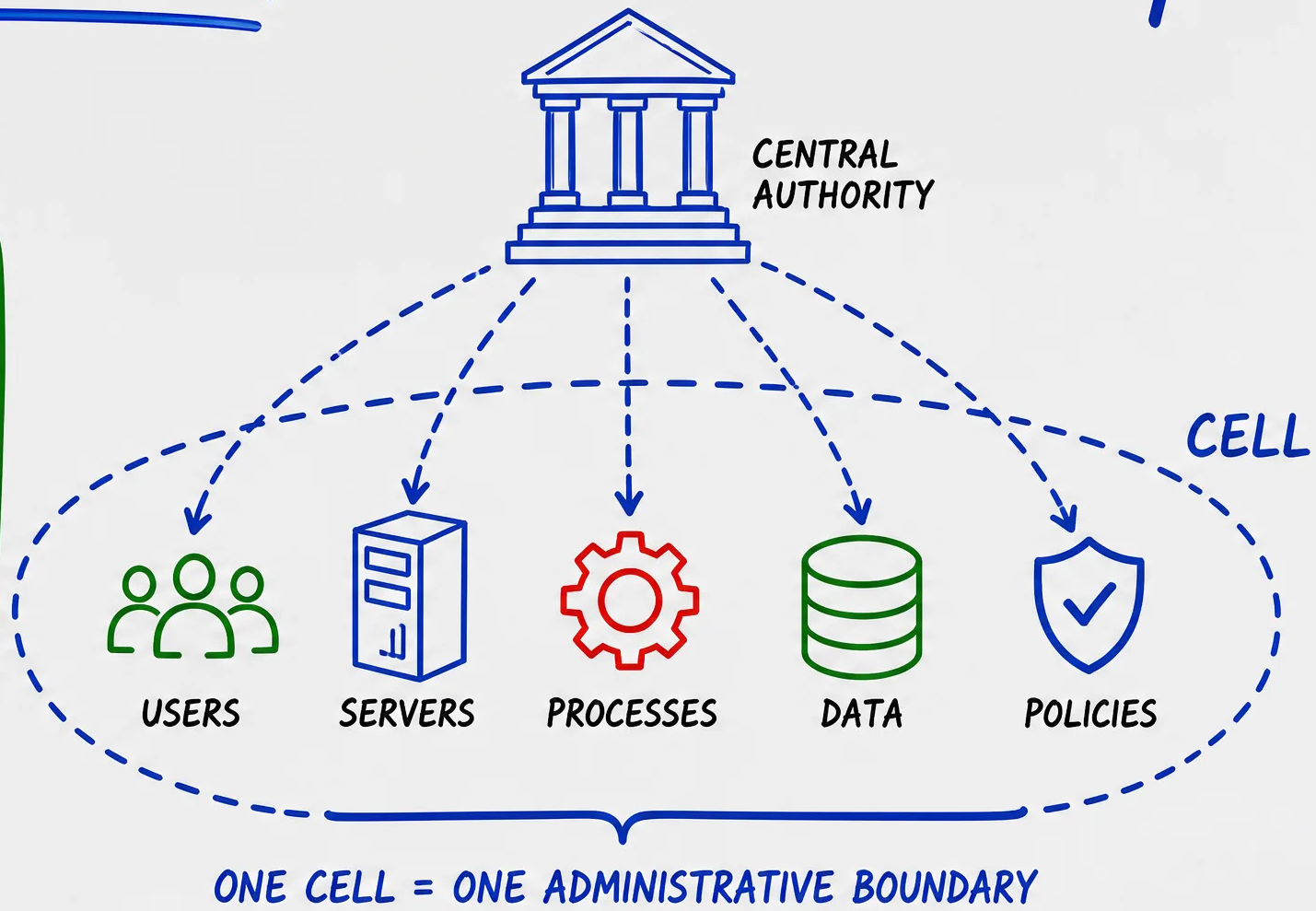
1. **NODE INITIATES FEDERATION**
   The node connects to the cell and requests to join.
2. **CELL VALIDATES**
   The cell verifies the node's identity, version, and configuration.
3. **TRUST IS ESTABLISHED**
   The cell issues credentials and trust is established.
4. **NODE BECOMES CELL MEMBER**
   The node is added as a managed member of the cell.
5. **MANAGEMENT BEGINS**
   The cell can now deploy, monitor, and manage the node.



FEDERATION **HANDS THE KEYS** TO THE DEPLOYMENT MANAGER.

Distributed: One Cell, Central Authority

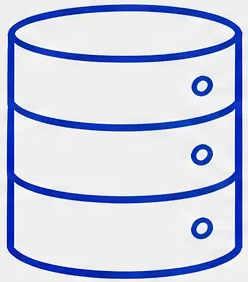
The cell is the **ultimate** administrative boundary.



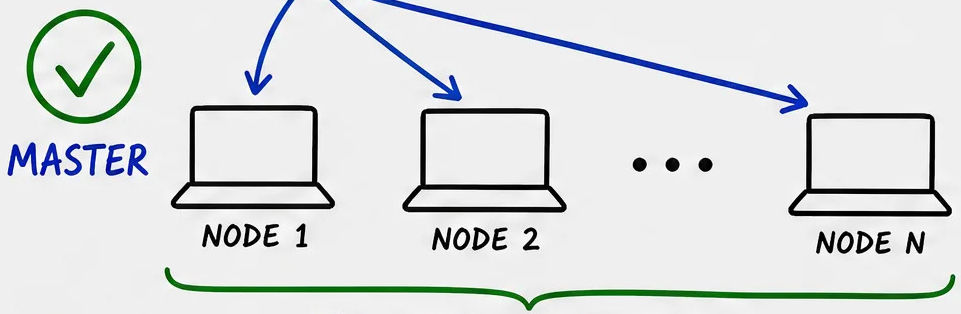
One Master, Targeted Local Copies

The master configuration dictates reality.

MASTER CONFIGURATION REPOSITORY



One source of truth.
Consistent, controlled, and versioned.
Changes made once, applied everywhere.

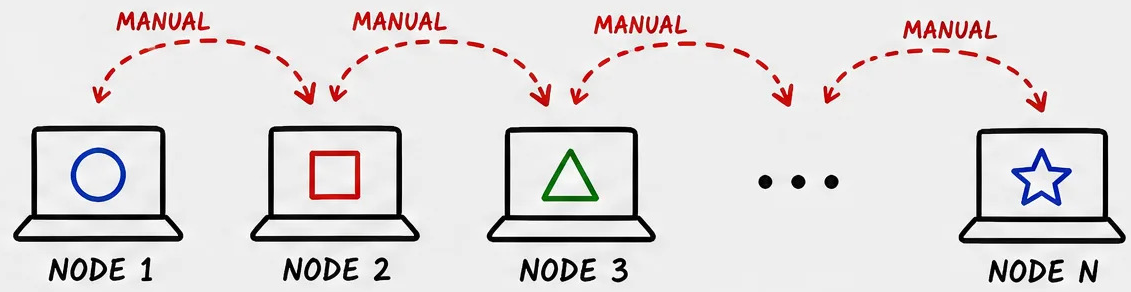


TARGETED LOCAL COPIES
Up-to-date. Consistent. Reliable.

NODE-LOCAL CONFIGURATIONS



Multiple sources of truth.
Drift, inconsistency, and manual effort.
Changes must be repeated everywhere.



DRIFT • INCONSISTENCY • RISK

Where the Configuration Lives

Identify configuration file locations and the cell hierarchy.

CONFIGURATION FILE LOCATIONS

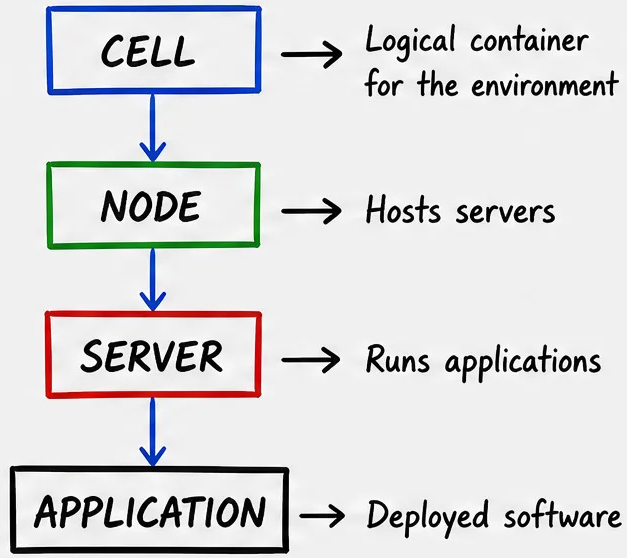
Cell Root
→ <cell_root>/config/cell.xml

Node
→ <cell_root>/nodes/<node>/node.xml

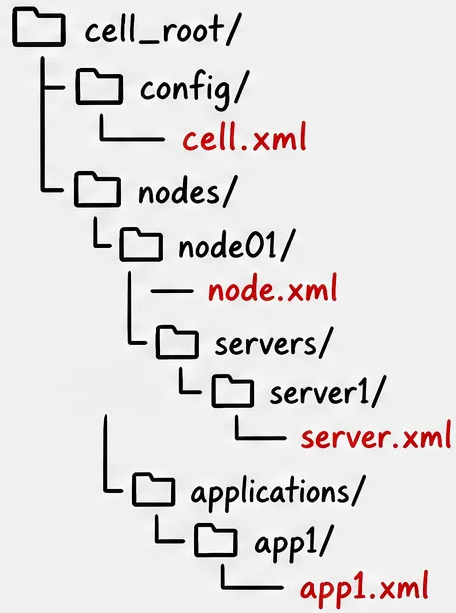
Server
→ <cell_root>/nodes/<node>/servers/<server>/server.xml

Application
→ <cell_root>/applications/<app>/<app>.xml

CELL HIERARCHY



FILE SYSTEM VIEW



The file system proves what the repository believes.

Scope Resolves from Broad to Specific

CONFIGURATION SCOPE

When the same setting is defined at multiple levels, the most specific scope wins.



A local setting outranks a global setting.



MOST SPECIFIC

CELL Affects a single cell (local).

NODE Affects all cells on a node.

SERVER Affects all nodes on a server.

MOST BROAD



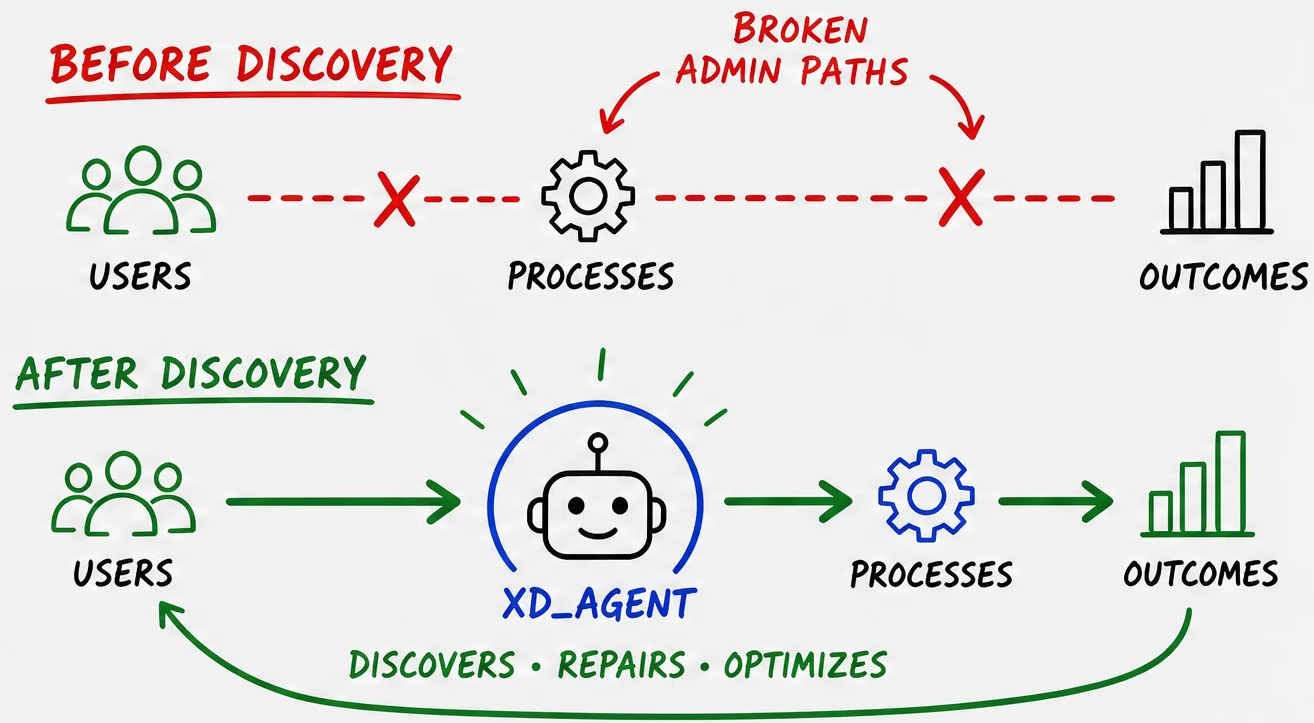
HIGHEST PRIORITY

LOWEST PRIORITY

DISCOVERY LETS MANAGEMENT PROCESSES RECONNECT

HOW XD_AGENT DISCOVERS

1. XD_AGENT continuously scans the environment.
2. Identifies process relationships, owners, policies, and flows.
3. Detects broken or missing administrative paths.
4. Automatically remediates by reestablishing and optimizing connections.
5. Keeps the model current as things change.



DISCOVERY REPAIRS BROKEN ADMINISTRATIVE PATHS **AUTOMATICALLY.**

Synchronization Is a One-Way Consistency Service

Configuration Synchronization: Dmgr → Node



DMGR

Deployment Manager

PUSHES CONFIGURATION

- ✓ Applications
- ✓ Resources
- ✓ Policies
- ✓ Settings



NODE

Node Agent

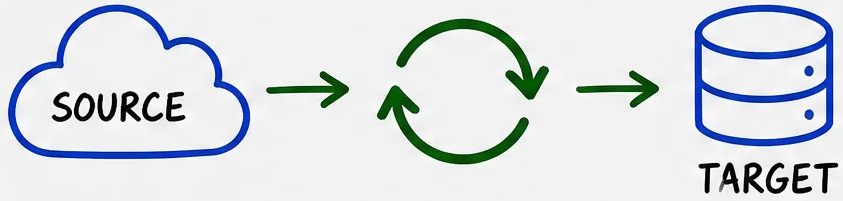
- Receives configuration
- Applies changes
- Maintains consistency
- Reports status



The deployment manager **pushes reality** to the node agent.

Set Synchronization for the Operational Context

AUTOMATIC SYNCHRONIZATION



- ✓ One-click / scheduled sync
- ✓ Fast propagation of changes
- ✓ Ideal for development & testing

VS.

MANUAL SYNCHRONIZATION

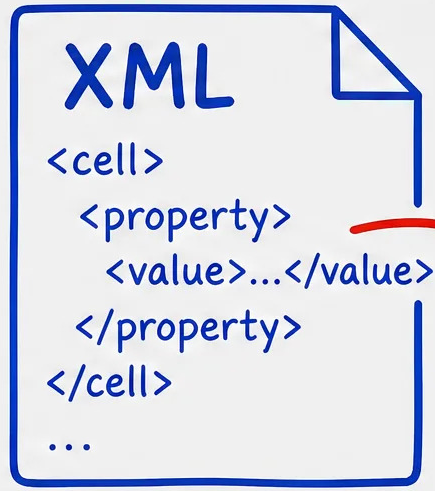


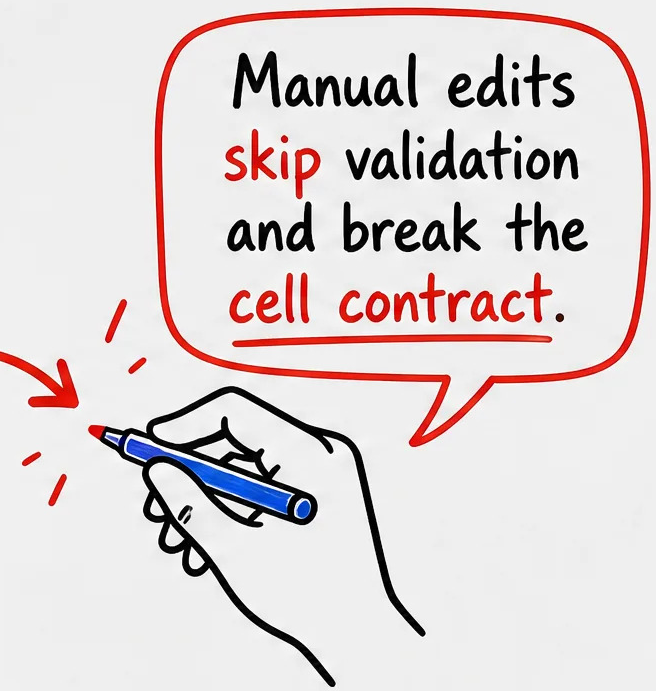
- ✓ Intentional, controlled sync
- ✓ Reduces risk of accidental impact
- ✓ Ideal for production environments

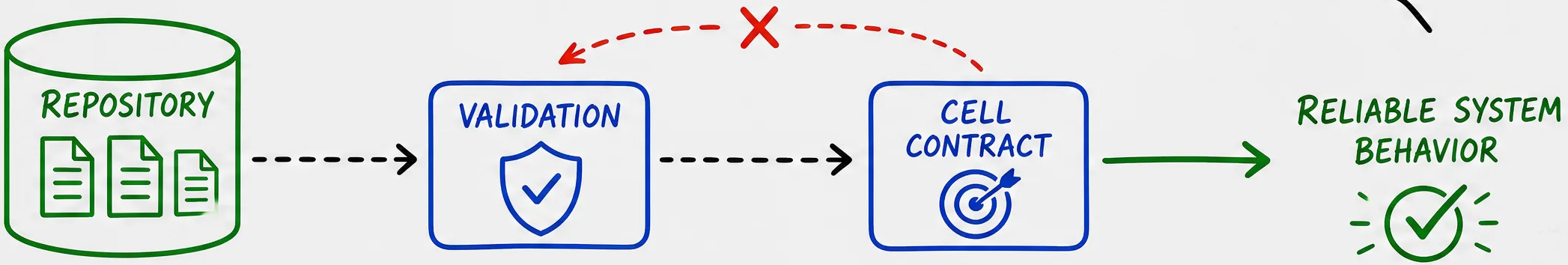
Automatic sync speeds up dev; manual sync protects production.

Manual Repository Edits Are a Recovery Exception

Do NOT edit repository XML files by hand.

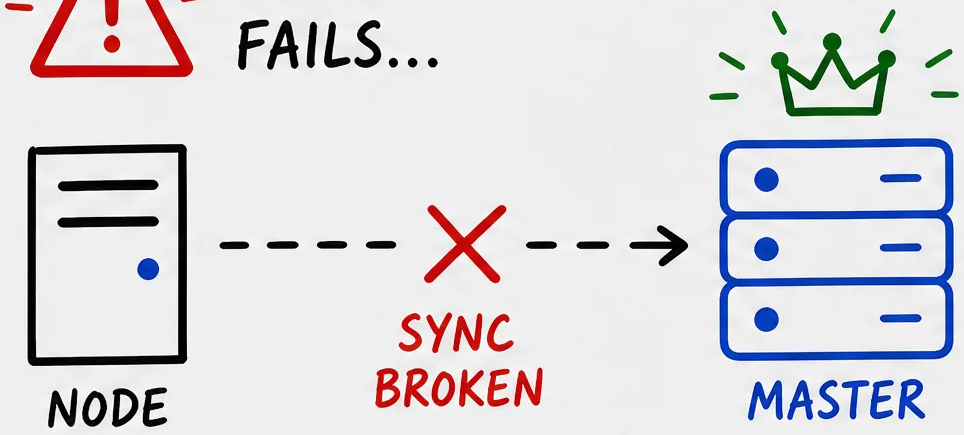






Choose the Synchronization Recovery Action

WHEN SYNCHRONIZATION FAILS...



USE syncNode TO RECOVER



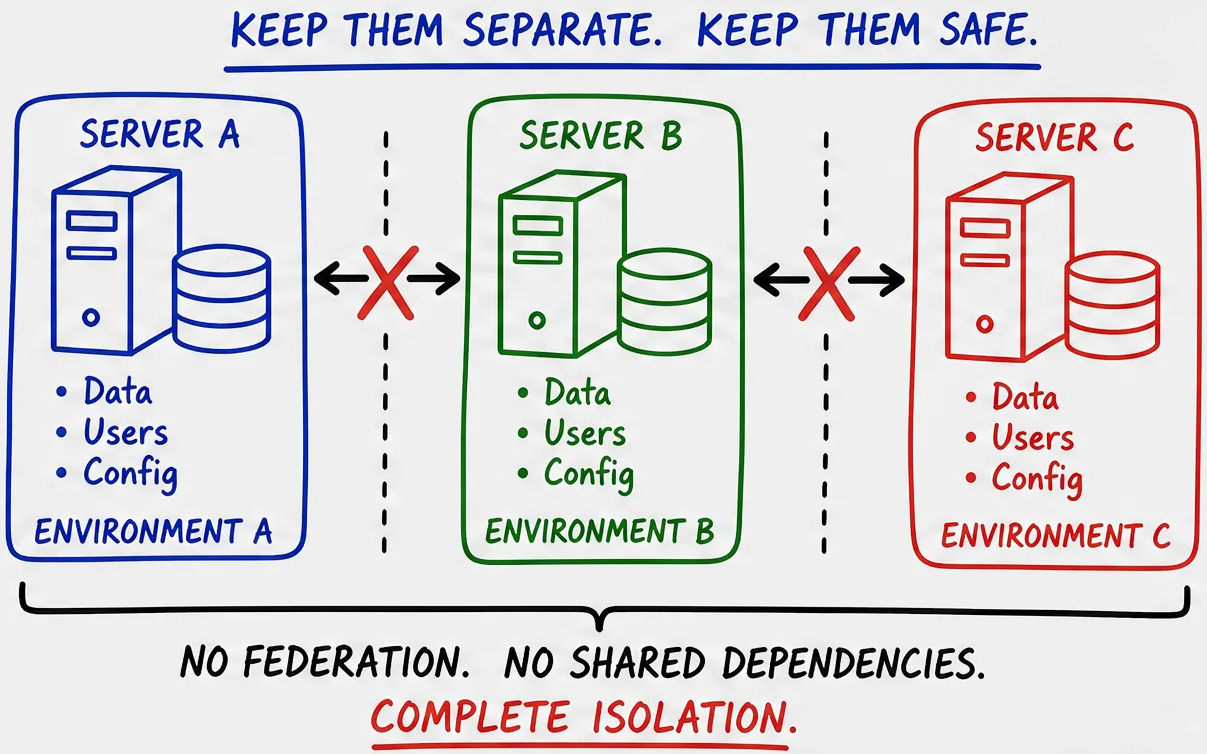
When automatic synchronization breaks, syncNode forces a reset from the master.

Multiple Stand-Alone Servers: Why Keep Them Separate?

Organizations keep multiple stand-alone servers un-federated to achieve complete isolation across environments.

KEY REASONS

- **Security & Compliance:** Strict data boundaries, regulatory separation, and reduced blast radius.
- **Independence:** Autonomous operations, upgrades, and maintenance without cross-impact.
- **Environmental Separation:** Different regions, tenants, business units, or classifications.
- **Simplicity by Design:** Smaller scope, fewer dependencies, easier troubleshooting.



Complete isolation is an **architectural choice**, not a **configuration accident.**

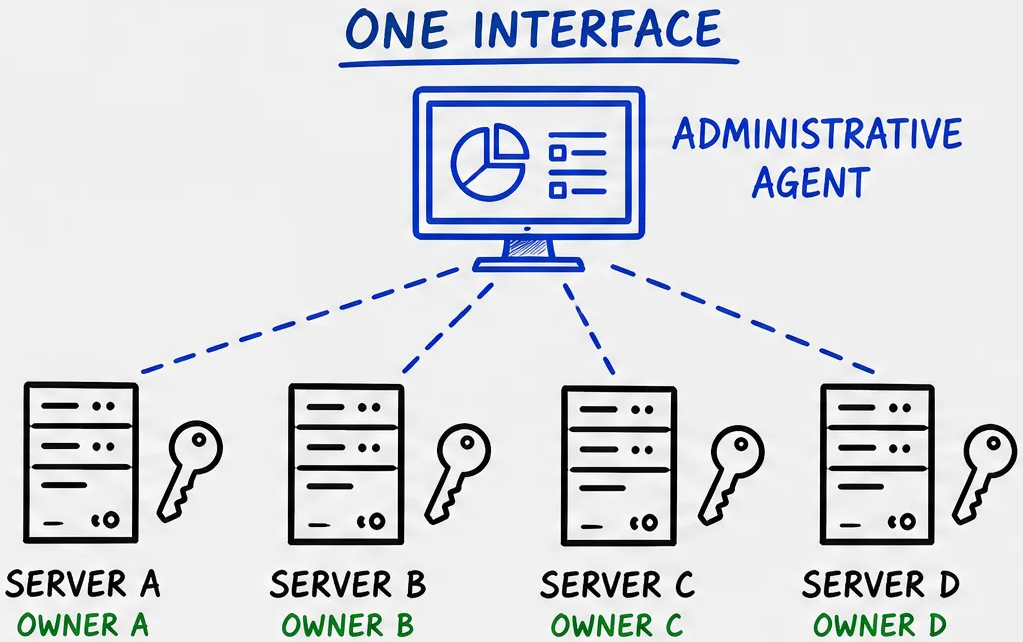
Administrative Agent: One Interface, Separate Ownership

INTRODUCING

The Administrative Agent

A single interface for managing un-federated servers—without taking ownership.

Centralize management. Preserve autonomy.



KEY BENEFITS

- Unified visibility and controls
- Ownership and control stay with each owner
- Simplify operations, without sacrificing autonomy

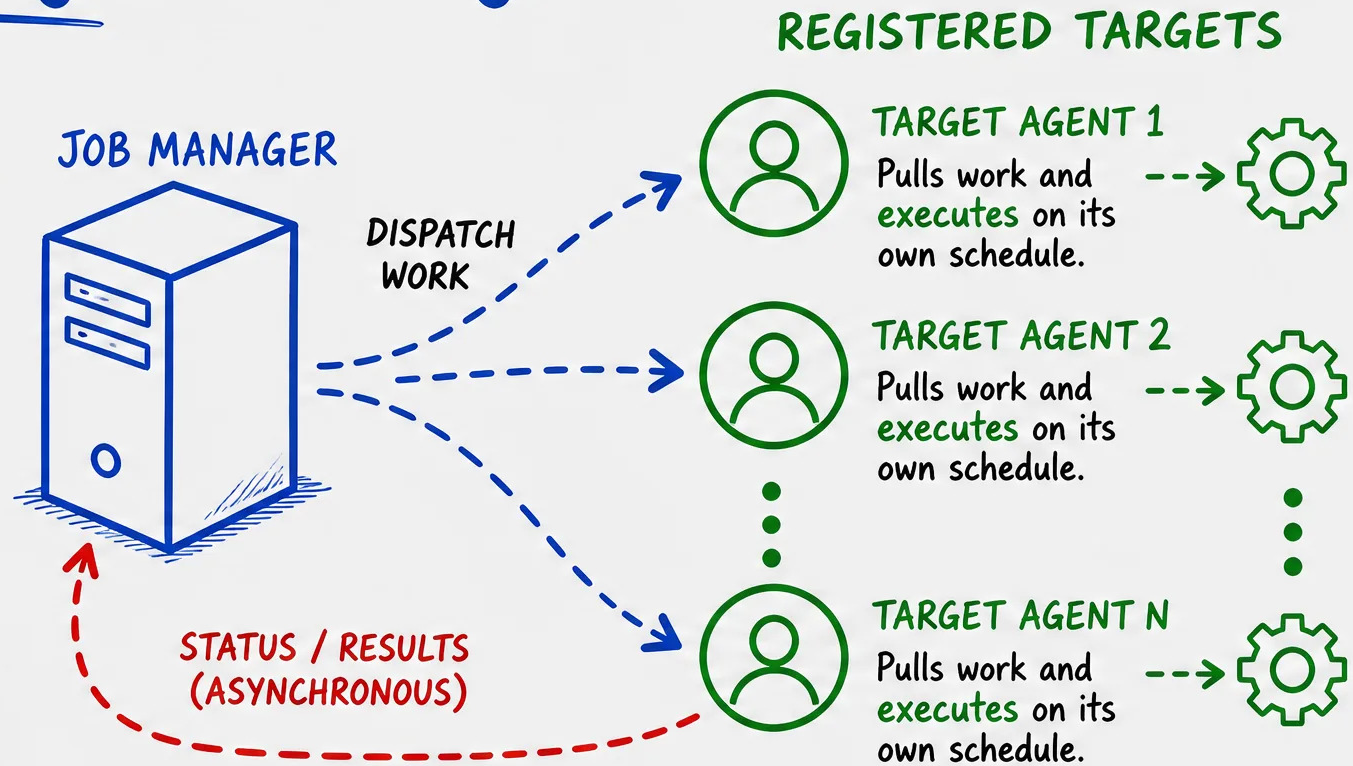
An administrative agent unifies the console without giving up the keys.

Job Manager:
Asynchronous Work Across Registered Targets

The job manager **sends work**, but target agents **control execution.**

KEY TAKEAWAYS

- Job Manager coordinates and dispatches work.
- Registered target agents pull and execute work on their own schedule.
- Asynchronous model improves scalability, flexibility, and reliability.

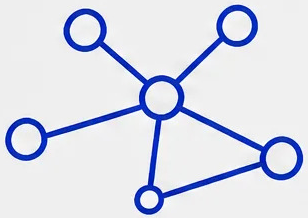


SCALABLE | FLEXIBLE | RELIABLE

Operator Decision Walkthrough

The topology decides the tool, not the other way around.

1 UNDERSTAND THE TOPOLOGY



- Map users, processes, data, and dependencies.
- Find constraints and connections.

2 DEFINE THE OUTCOME



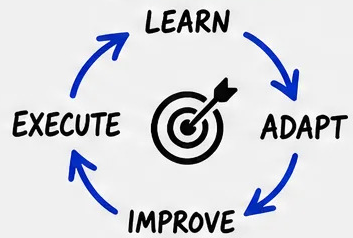
- Clarify the goal.
- Identify success measures.

3 MATCH THE TOOL



- Choose the tool that fits the topology.
- Balance capability, risk, and effort.

4 EXECUTE & ADAPT



- Run, measure, learn, and adapt.
- Adjust as topology or needs change.

5 DELIVER VALUE



- Realize outcomes.
- Drive impact and continuous value.

REMEMBER: Great operators start with topology, choose the right tool, and adapt for results.

OUTCOME:
Better decisions.
Lower risk.
Stronger results.

Recap — Think Like the Management Architecture

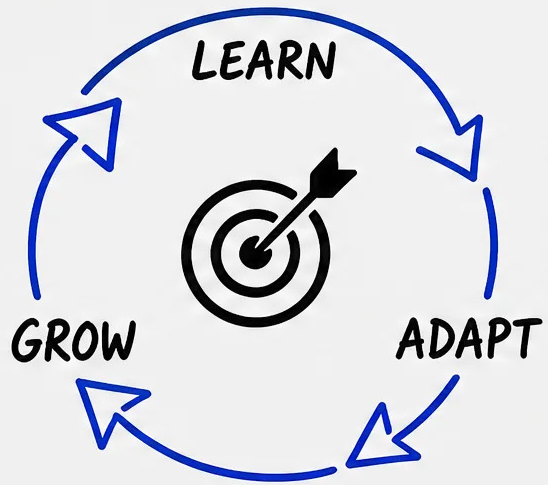
You administer the hierarchy; WebSphere runs the servers.

YOU ADMINISTER



- ✓ Users
- ✓ Groups
- ✓ Roles
- ✓ Policies
- ✓ Processes
- ✓ Outcomes

DEFINE. GOVERN. EVOLVE.



WEBSPHERE RUNS THE SERVERS



EXECUTE. SCALE. PROTECT.



You set the strategy. WebSphere delivers the execution.